

ASSESSMENT 2 OF 2 · CONTINUOUS ASSESSMENT

A2 — Applied AI System

Group · 20% of your course grade · Marked by the instructor

Released	Sunday 30 August 2026	Weight	20%
Due	Sunday 13 September 2026, 23:59 SGT	Covers	Weeks 4–6
Team declaration	Friday 4 September 2026, 23:59 SGT	Rubric	Rubric 1 (four criteria)
Submit to	NTULearn	Teams	Your existing course group

What A2 is, in one paragraph. Class 4 argued that *the harness is the deliverable* — the agent is the easy half, and what separates a system you would deploy from a demonstration that got lucky is the evidence around it. A2 marks that claim — and it does not stop at Class 4. **A2 runs across Weeks 4, 5 and 6, and it takes something specific from each. From Class 4:** the loop, the tool layer, the guardrails, the evaluation set. **From Class 5:** the three-layer cost model — per-task variable, expected fallback, fixed monthly — priced at your problem's real volume, with a sensitivity range instead of a point estimate and a break-even success rate for the cheap model. **From Class 6:** the guardrail checklist and the attack surface, because every one of these problems reads text written by someone outside your organisation.

Read this first if you are one of the non-technical two-thirds. Every team has at least one member who codes, and that is deliberate — but **it does not mean the coder does A2 and everyone else watches.** Look at where the marks are: Conceptual Understanding (25%) and Reasoning & Justification (25%) are **half the total**, and almost none of it is code. D0 is writing. Choosing the tool set and writing the six-field descriptors in D2 is design. Writing evaluation cases is domain judgement — knowing which claim *should* be refused. Reading the cost model and saying which lever mattered is business analysis. **A team where the coders code and nobody else engages will produce a working agent and a weak submission.**

Read the three weeks as one argument. Class 4 asks *does it work?*, Class 5 asks *what does it cost when it does not?*, Class 6 asks *what happens when someone attacks it?* Your submission has to answer all three about the same system.

A modest agent with a real evaluation set, real caps, a defended autonomy setting and an honest cost model will score well above an impressive agent with no evidence.

WHAT IS IN THIS BRIEF

Sections 1 to 3 are what you do. Section 5 is how it is marked — read it before you start. Appendix A has both problems in full.

1 · CHOOSE YOUR PROBLEM	2
2 · WHAT WE PROVIDE, AND WHEN	2
3 · WHAT YOU BUILD	4
D0 · Why an agent at all	4
D1 · The agent	6
D2 · The tool layer	8
D3 · The guardrail layer, and the guardrail checklist	11
D4 · The evaluation set	11
D5 · The model battery	13
D6 · The cost-to-serve model	14
D7 · Two reproduced failures	17

4 · WHAT YOU SUBMIT	18
5 · HOW A2 IS MARKED	19
6 · USING AI TO BUILD AI	19
7 · BUDGET	19
8 · TEAMS, CONTRIBUTION, AND THE FREE-RIDER CLAUSE	20
9 · WHAT FEEDBACK YOU GET, AND WHEN	21
10 · TIMELINE	21
11 · WORDS THAT DO REAL WORK IN THIS BRIEF	22
PROBLEM A · Health-insurance claim first response	24
PROBLEM B · Outpatient referral coordination	26

1 · CHOOSE YOUR PROBLEM

You do **not** define the problem for A2. Pick **one of two**. (The End-of-Course Project is where you define your own — A2 holds the problem fixed so the marks fall on how you build and how you prove it.)

Both share the same shape, which is the shape Class 4 taught: a request arrives, the agent must **go and look things up before it concludes**, and then reach **one of three outcomes** — act, ask for something missing, or hand to a human. Both contain cases where acting would be wrong, and in both the number of steps a case needs depends on what the agent finds.

	Problem	Domain	The gated action
A	Health-insurance claim first response	Insurance operations	Issuing the decision letter
B	Outpatient referral coordination	Hospital operations	Booking a clinic appointment

Full statements are in **Appendix A. Read both before choosing**. They are matched in difficulty and both genuinely need a loop; they differ in what makes them hard — Problem A in the volume of checks a single claim can demand, Problem B in the cost of acting when you should not.

There is no opt-out and no bring-your-own-problem route. If you want to build against something of your own, that is exactly what the End-of-Course Project is for.

A2 MAY NOT BE BUILT ON ANY MEMBER'S END-OF-COURSE PROJECT

- You may **not** submit for A2 a system, dataset, evaluation set or write-up that is part of any member's individual Project.
- You may **not** later submit A2 work — the agent, the harness, the fixtures, the report — as part of your own Project.
- Techniques travel freely. **Artefacts do not**. Nothing may be marked twice.

If your Project idea resembles one of the two A2 problems, that is fine and it is not a conflict — but the two builds must be separate work, and the Project must remain your own individual submission.

2 · WHAT WE PROVIDE, AND WHEN

What	When
This brief and the FAQ (Frequently Asked Questions)	NTULearn, from release — Sun 30 Aug
Team self-appraisal form	NTULearn, from release — Sun 30 Aug
Team declaration template — TEAM_DECLARATION.docx, the section 8 checkpoint due Fri 4 Sep	NTULearn, from release — Sun 30 Aug
Fixture-data generator — writes the starting data files for your problem, so no team spends its first evening inventing records	NTULearn, by midday Wed 2 Sep
Starter scaffold notebook — vendor-neutral configuration, the Class 4 loop extended to parse more than one action per turn (D2c), a tool-contract template, an skeleton evaluation harness for you to fill in, the cost cells	NTULearn, by midday Wed 2 Sep
Already yours — the Class 4 Capsule 2 notebook is a working agent with five tools, guardrails and an eval harness. It is the skeleton, and starting from it is expected and not penalised.	You have it

The scaffold is a starting point, not a specification. It exists so nobody loses a day to setup code. **Change anything in it you like** — rename it, restructure it, replace whole cells, throw parts away, write your own from scratch if you prefer. Nothing in the marking rewards keeping our code intact, and nothing penalises departing from it. **The only thing that matters is that your system meets the A2 objectives in this brief.** If the scaffold's shape fights your problem, the scaffold is wrong, not your problem.

The fixture data is a seed, not a fixed set — extend it. The generator writes a starting set of records and, more importantly, **shows you the pattern:** what each file contains, what the fields are called, and **how the files connect to one another.**

That last one matters most, because it is how your agent gets from one fact to the next. In Problem A a claim record carries a member_id, and the members file is keyed by that same member_id — which is what lets lookup_policy take the member_id that get_claim returned, find that member, and follow their policy_id to the policy that covers them. Invent a claim whose member_id matches nobody and your agent will look perfectly sound and return nothing.

Add your own records freely, following that pattern. You will have to: a 30–50 case evaluation set with 6–10 negative cases needs records that trigger those negatives, and inventing them deliberately is part of the work. Two conditions only — **keep the records we supplied** (a marker re-runs your harness against them), and **commit whatever generates or holds your additions**, so the data set is reproducible rather than a mystery.

The records we supply come with a labelled answer key. For every record in the starting set, expected_outcomes_A.json / _B.json gives the outcome the routing table in Appendix A requires and **the single trigger that produces it.** That is your ground truth for D4, and it is the shape your own labels have to take for the cases you invent. **A run that reaches the right outcome by the wrong trigger is not a pass** — it got there by luck, and it will not get there next time.

FOUND A BUG? TELL ME, AND IT COUNTS

The generator and the scaffold were written for this assignment and have not been through a hundred hands. **If you find a defect — data that contradicts itself, a field that is never populated, a function that does not do what its own description says it does, a case that cannot be satisfied as written, an instruction in this brief that contradicts another one — report it.**

Email me directly, and early. A defect found on 2 September helps 27 other teams; the same defect reported on 12 September helps nobody, including you.

Bugs I acknowledge earn credit under Class Participation. This is not a token gesture: noticing that a system's own documentation does not match its behaviour is precisely the skill this course spends six weeks arguing for, and it is the same instinct that makes a good evaluation set. Report what you found, where, and what you expected instead — the same shape as a good bug report anywhere.

You are not required to wait for a fix. Say in your repository what you found, what you assumed instead, and carry on. A documented workaround is a strong answer, not a compromised one.

You are not blocked until Wednesday. D0 is writing, not coding, and the tool-set and descriptor work in D2(a) and D2(b) is design. Start there — it is the order this brief asks for anyway.

Your US\$10 OpenRouter key is the one you already hold. There are no new credits for A2.

3 · WHAT YOU BUILD — EIGHT REQUIRED PIECES

Everything below is required. The order is the order to build in.

D0 · Why an agent at all — answered before you write any code

This is the first thing we read, and it is the first section of your report. Class 4 opened with it and so did its notebook. A team that skips it and goes straight to building has already lost marks in two criteria.

D0(a) · Place your problem on Class 4's ladder, and defend the rung

Class 4 Capsule 1 put **seven rungs** on the board, cheapest first, from Anthropic's Pre-read 1. **Rungs 1 to 6 are workflows — predictable, enumerable, testable. Only rung 7 hands the sequence to the model.**

Rung	Who decides	Cost	Use it when
1 · Single call	you	1 call	one-shot, well-specified task
2 · Prompt chain	you	2–4 calls	fixed subtasks with a check between
3 · Routing	model picks a lane	1 + 1 calls	distinct categories; cheap model for the easy ones
4 · Parallelisation	you	N calls, one wall-clock	independent checks, or a vote
5 · Orchestrator–workers	model, at runtime	unpredictable	the split depends on the input
6 · Evaluator–optimiser	model	unknown rounds	criteria statable; a second pass helps
7 · Agent	model	unbounded until you cap it	steps cannot be known in advance

Class 4's rule, unchanged: *go down the list only when the rung above genuinely cannot do the job. A2 sits on rung 7.* Your task is not to justify choosing it — the brief chose it. Your task is to say **what rungs 1 to 6 would and would not have delivered on your problem**, and what rung 7 cost you.

Answer these, in writing.

1 · **The workflow test.** Four questions separate a workflow from an agent:

The question	Workflow	Agent
Who decides the sequence of steps, and when?	decided in advance, by you, in code	decided in the moment, by the model
Does the number of steps vary with the input?	no — the same path every time	yes — that is the whole point
Can you test every path?	yes, they are enumerable	no — you test outcomes, not paths
What does it cost?	predictable: n calls	unpredictable until you cap it

The second row is the one that decides it. *If the number of steps does not vary with the input, you want a workflow — however clever the pitch sounds.* Show, with cases from your own evaluation set, that yours genuinely varies.

2 · **The two conditions.** An agent is a system where the model “*dynamically directs its own processes and tool usage, maintaining control over how it accomplishes tasks.*” Two things must hold — take either away and it is something else:

- **The steps are not known in advance.** Without this, you have a **workflow**.
- **It gets ground truth back at every step** — a tool result, a query, code that ran — so reality can correct it. Without this, you have a **monologue**: fluent, self-consistent and unfalsifiable.

3 · **The three-question test — where the governance cliff is.**

	Who picks what to retrieve?	Can it loop and re-query?	Can it change the world?
Retrieval (RAG)	your code — one fixed query	no — a single pass	no — read-only

	Who picks what to retrieve?	Can it loop and re-query?	Can it change the world?
Agentic retrieval	the model, at runtime	yes — it re-queries on what it finds	no — read-only
Agent	the model, at runtime	yes	YES — it writes, sends, books, pays

Agentic retrieval is simply an agent whose tools are all read-only. The governance cliff is not retrieval → agentic retrieval. It is agentic retrieval → agent, at the first write. Name **your** first irreversible action; that write is the whole reason your problem is on rung 7 rather than rung 5 or 6.

D0(b) · When NOT to build an agent — both Capsule 1 tests, answered honestly

Capsule 1 gave two tests, and said a task failing **either** one should be built as the rung below. Answer both.

Test 1 · The ground-truth test. *What will tell this loop it is wrong, and how fast?* A failing test suite answers in seconds and cannot be argued with; a customer's satisfaction answers in days and is a matter of opinion. **Fast and objective → build the agent, wiring that signal in first. Slow or subjective → a workflow with a human gate.** And if there is no signal at all, *you do not have an AI problem, you have an unspecified one — no amount of autonomy fixes an objective nobody can state.*

Name the **systems of record in your problem that can contradict the model within seconds** — the policy row, the slot table, the document status — and say how fast each answers. Class 4's worked answer was an order row, a carrier scan and an inventory count: *ground truth arriving at machine speed*, which is the whole reason a loop is allowed to run unsupervised.

Test 2 · The arithmetic — and this one feeds the rest of the assignment.

Read the exponent carefully: it is the number of steps in ONE run, not the number of cases in your evaluation set. Class 4 asked what happens when a *single* trajectory has to get twenty things right in a row. Nothing here is about how many cases you test.

Per-step reliability compounds *within a run*. If each step succeeds **95%** of the time and one run takes **20** steps:

$0.95^{20} \sim 0.36$ one run in three completes with every step correct
 $0.99^{20} \sim 0.82$ even at 99% per step

Capsule 1 gave three ways out, and **the middle one is the biggest win**: raise per-step reliability (hardest) · **cut the number of steps (biggest win)** · make failure recoverable.

Now do it with your own numbers — and note which direction you are working in. You cannot measure per-step reliability directly; what you measure in D4 is whether a *whole run* produced the right outcome. So you work backwards. If your measured run pass rate is **P** and your measured median turn count from D7 is **T**, the per-step reliability that would produce that result is:

implied per-step reliability $s = P^{(1/T)}$
 Illustrative: $P = 0.78$ measured over a median $T = 6$ turns
 $s = 0.78^{(1/6)} = 0.959$ each step goes right about 96% of the time
 at $T = 3$ $\rightarrow 0.959^3 = 0.88$ cutting to three turns predicts ~88%
 at $T = 12$ $\rightarrow 0.959^{12} = 0.61$ wandering to twelve predicts ~61%

That is the whole argument in three lines. The same agent, the same per-step quality — and run success anywhere between **61% and 88%** depending on turn count alone. **Way 2 is D2(c)** — parallel tool calls collapse turns — **and the run pass rate you just moved is exactly what sets layer 2 of your cost model in D6**, because layer 2 is $(1 - P) \times \text{failure_cost}$. These are not three separate exercises. They are one argument, measured three times.

Not every step has the same s — and finding the weak one is the point of the exercise. The turn that reads a policy row is near-perfect. The turn that judges a member's free-text narrative is not. A single number averages them and hides which is which.

Your trace log already holds the answer. Group your failing runs by which tool call came immediately before things went wrong. The turn that shows up most often before a wrong outcome is your candidate.

Then notice you have two different moves, and they are not the same lever.

	The move	Which of Class 4's three ways	What it changes
(a)	Fix that step — a better tool descriptor, a filtered return, a tighter type	Way 1 · raise per-step reliability	s goes up, so P goes up
(b)	Remove or bypass that step — cut the tool, move the work into ordinary code, or fold it into a parallel turn	Way 2 · cut the number of steps	T goes down, so P goes up

And here is the connection worth putting in your report: a weak step usually costs you *both*. When a turn returns something poor, the agent tends to re-read, retry or wander — so an unreliable step **depresses s and inflates T at the same time**. That is why finding it repays the effort: one fix moves both terms. It is also why your D7 loop failure and this arithmetic are the same investigation looked at from two ends.

State the limits, because they are real. Steps are not independent — a bad observation early makes later steps worse, not equally likely to succeed — and not every step is equally failure-prone; the tool call that reads a policy is more reliable than the one that judges a free-text narrative. **So treat s as a diagnostic, not a physical constant of your system.** It is a way of asking “*is my problem step quality, or step count?*” and the answer changes what you should work on. Say in the report which of the two your evidence points at.

D0(c) · What good looks like — written before any agent code

Five numbered statements about a good run, in the repository, **committed before your first agent commit** (we will look at the commit history). Class 4's example, to copy the shape of:

- 1· Names the **real** cause, traceable to a record — not a plausible story.
- 2· Gives an outcome **consistent with** what the records actually say.
- 3· Takes the gated action **at most once**, and only after the facts are established.
- 4· Says “**I don't know**” rather than inventing an answer the records do not support.
- 5· Costs less than a person doing it.

Item 4 is the one teams forget, and it is the one your negative cases exist to catch. Your evaluation set in D4 is downstream of this list — if a statement here is not testable, rewrite it until it is.

D1 · The agent — and the architecture you are asked to build

A2 is a single-agent ReAct (Reason + Act) build. That is the architecture, and it is not a default you may upgrade away from.

One agent. One control loop. Thought → Action → Observation → repeat → Final. A tool-using loop over a foundation model: it reads a request, decides which tool or tools to call, reads the results, decides again, and stops with an outcome. Class 4's notebook is a working skeleton you may start from, and starting from it is not penalised.

Your agent must use several tools, and it must be able to call more than one of them in a single turn. Class 4's loop deliberately reads one Action: per turn; A2 asks you to extend that. Why, and how, is D2(c). **Calling three tools in one turn is still one agent** — it is not multi-agent, and nothing in D2(c) changes the architecture.

Out of scope — and this is a scope rule, not a difficulty rule

Not in A2	Why
Multi-agent systems — orchestrator-and-workers, agents that call agents, specialist agents handing off	Class 4 Capsule 1 compared the patterns. Capsule 2 built exactly one of them. We do not mark what we did not teach you to build.
Sub-agents as a way to widen your tool set	Same reason. Listed in the abstract, never built, and it would hide the loop A2 is marking.

Not in A2	Why
A framework that owns the loop — LangChain, LangGraph, CrewAI, AutoGen and relatives	Class 4 hand-rolled the loop on purpose and named “the framework trap”. The loop is the thing being marked. If a library runs your loop, there is nothing of yours to mark in D2(c) or the guardrail layer. Ordinary libraries for everything else — data handling, HTTP, testing, plotting — are fine and expected.
Fine-tuning a model	Not taught, not affordable on a US\$10 key, and not what this assignment is about.

None of this is a ban on thinking about them. The opposite: **argue about them in your report and it earns marks.** ILO (Intended Learning Outcome) 3 is about evaluating design alternatives, so a paragraph saying “*a second agent reviewing our decision letter would have caught failure X — here is what it would have cost, and here is why we did not build it*” is exactly the reasoning we want. Pre-read 5 gives you the evidence: Cognition shipped Devin on a multi-agent claim, then published its own correction — “**Don’t Build Multi-Agents**” (June 2025) and “**Multi-Agents: What’s Actually Working**” (April 2026), in which the review agent is multi-agent but **writes stay single-threaded**.

In short: build one ReAct agent. Reason about the alternatives in prose. Marks for the argument, none for the unbuilt architecture.

The scope boundary — read this before you plan any work

Your tools operate on **fixture data**: small files on your own machine — JSON (JavaScript Object Notation), CSV (Comma-Separated Values, which opens in Excel), or a small SQLite database file if someone on your team already works that way. **No live systems and no real accounts** — nothing your agent touches may be somebody’s actual record.

A tool that reaches the public internet — optional, and conditional. You do not need one. Every problem here is answered from its own records, no marks depend on having a search tool, and the shortest defensible tool list rarely contains one. **But if you want one, one condition applies and it is not negotiable: your scripted run must still reproduce with no network and no key.** Call the service once, **commit the responses as fixtures, and have the scripted backend replay them.**

That is ordinary practice for testing anything that talks to a network, and it is the only reason D5(a) can be a requirement at all. **An evaluation set that cannot be re-run because the web moved underneath it is not an evaluation set** — the same case would pass on Tuesday and fail on Thursday for reasons that have nothing to do with your agent. Record and replay, or leave it out.

AND THE GATED ACTION IS A LOG ENTRY, NOT THE THING IT STANDS FOR

This is the single most common way a team could waste a week of A2, so it is spelt out. **The gated action is one function, three steps:** (1) check the gate — is the autonomy setting satisfied, has the operator confirmed where your setting requires it; (2) append one structured record to a local file; (3) return a confirmation string the agent can put in its final answer. **That is the whole write.**

Something of this shape is a full-marks implementation:

```
{
  "ts": "2026-09-08T14:22:07",
  "case_id": "CLM-8842",
  "decision": "approve_in_principle",
  "reason": "policy live to 2027-03-31; all 3 lines covered; panel hospital",
  "evidence": [
    "get_claim",
    "lookup_policy",
    "check_coverage x3",
    "get_preauthorisation",
    "get_hospital_status"
  ],
  "autonomy": "confirm",
  "gate": "operator approved at turn 4",
  "turns": 4,
  "cost_usd": 0.0041
}
```

What you are NOT being asked to build. None of the following earns a single mark, and every one is a way to run out of time:

A letter or document generator	A booking or appointment application	A user interface of any kind
--------------------------------	--------------------------------------	------------------------------

PDF or Word output	A calendar or scheduling integration	A web front end
Real email sending	A database server	Anything deployed anywhere
Templated customer prose	A login or user accounts	A mobile app

Now look at what is NOT in that record. No letter text. No greeting, no recipient address, no policy summary, no template. Nothing was sent anywhere. **The record is the output of your agent** — it is the thing your system produced, and it is the thing that gets marked.

The names are the business action; the code is a log line. `issue_decision_letter` does not write a letter. `book_slot` does not book anything. `record_decision` writes a row. They are named for what they *stand for* so the governance argument stays legible — a tool that writes makes the system an actor, whichever file it writes to. **An email application, a booking screen or a document generator would be a different assignment**, and building one would consume the fortnight without moving a single criterion in Rubric 1.

Where the marks actually are: in **which** decision the agent reached, **on what evidence**, **after which gate**, and **at what cost** — and in the harness that proves it across your evaluation set. A team whose record shows the right decision with a traceable evidence trail scores above a team that generated a beautiful letter for the wrong claim.

One place teams over-build. The “ask for something missing” outcome needs a **short string naming the missing item** — “*request: valid pre-authorisation for procedure 47120*”. It does not need customer-ready prose, a tone of voice, or a template. Nothing in Rubric 1 marks the wording of that string.

No tool may take a real-world irreversible action — no real email, no real payment, no real booking, no writing to any live system. This is a hard rule, not a shortcut. Class 4's notebook simulated its single write for exactly this reason, and the guardrail argument is unchanged by the fact that the write lands in a file.

D2 · The tool layer — the part that decides whether this agent is any good

Three parts. Class 4 called the tool layer the **ACI — the Agent–Computer Interface** — and made the point that the tool descriptions and signatures are *the entire manual the model gets*: it cannot ask a colleague, hover a tooltip, read your source, or try it in staging. Everything in D2 follows from that.

D2(a) · The tool set — chosen, not collected

Score **every** tool you ship against the three questions from Class 4, in a table in the repository:

Question	What a good answer looks like
1 · Does a task actually fail without it?	Name the task that fails. Start from the minimum set and add on an observed failure, never an imagined one.
2 · Could the model confuse it with a neighbour?	The driver is discriminability, not count . Ten obviously-different tools are fine; three overlapping ones are trouble. If you cannot say in one line when to use A rather than B, neither can the model.
3 · What does it cost when it is never called?	Its definition sits in the prompt prefix — the fixed block of text sent at the start of every single turn — so it is re-sent and re-billed every turn, called or not . It widens a failure surface you cannot test exhaustively. If it writes, it is one more gate.

Worked example — Class 4's own five tools, scored by the notebook itself. We use the Class 4 example rather than either A2 problem, so that no team gets its own answer handed to it. Copy the *format*, not the content.

Tool	Fails without it?	Confusable?	Why it earns its place
lookup_order	Yes — nothing else resolves an order id	No	The entry point; everything else needs its output
check_shipment	Yes — the only source of the real cause	No	The one tool that can contradict a plausible story
check_inventory	Yes, for the date — stock could make the promise wrong	No	Cheap, and it prevents a confidently wrong promise

Tool	Fails without it?	Confusable?	Why it earns its place
search_notes	No	Yes, with the two above	Kept deliberately as the trap. It fails questions 1 and 2 — and it is exactly the tool that causes the Section 5 failure. In production it is the one you would cut
send_customer_email	Yes — the task says “tell them”	No	The only write. One gate covers the whole agent

That the tool failing questions 1 and 2 is also the tool that breaks the agent is not a coincidence — it is the argument. Run the same three questions over your own set and see which tool comes out looking like `search_notes`.

Before you add a tool, try not adding one. Four moves, in order of preference — say in the report which you tried:

1. **Widen an existing tool's parameters** instead of adding a sibling.
2. **Return more from one call** instead of adding a second lookup.
3. **Move the step out of the loop** into ordinary code, before or after the agent runs.
4. Only then add the tool.

We credit the shortest defensible list. Four justified tools beat eleven. **A tool you removed, naming the observation that removed it, earns explicit credit** — that is the behaviour nobody does naturally.

D2(b) · Tool descriptors — write the manual, because it is the only one

Every tool ships a **descriptor contract**. Six fields, no exceptions:

NAME + SIGNATURE	<code>check_coverage(policy_id: str, procedure_code: str, site: Literal["SIN","KUL","HKG"]) -> CoverageResult</code>
WHAT	One line: what this answers that nothing else answers.
INPUT	Each argument, its type, and what a bad value does.
RETURNS	The shape, and a SIZE BOUND. "At most 3 records, 40 tokens each."
FAILS WHEN	The named conditions under which it returns nothing or errors.
IRREVERSIBLE?	Yes / No. If yes, name the gate that covers it.

Then apply at least two poka-yoke moves — design changes that make a class of error *impossible*, rather than asking the model to be careful. Class 4's table, to copy from:

Before	After	What it makes impossible
site: str	site: Literal["SIN-DC1", "KUL-DC2"]	A typo'd site silently returning “no record”
patient_name	patient_id	The wrong Tan
send_email(...)	draft_email(...) + send_email(...)	Sending when you meant to draft
dry_run: bool	dry_run: bool = True	The irreversible default

For each of your two, state **what it makes impossible** — not what it discourages.

And measure one rewrite. Take one tool, ship a **v1** and a **v2** of its descriptor and its return shape, and report tokens returned per call, evaluation pass rate, and guardrail cases passed, for each. This is Class 4 Section 7 applied to your own tool. The point it made, and the point we are marking: **a prompt instruction is paid for on every call of every run forever and dies when you change model; an interface constraint is paid once and holds.** If your v2 is not smaller, or not safer, say so — a rewrite that did not help, honestly reported, scores better than one that was never measured.

Run v1 on ONE model only — the cheap one — and quote it against your v2 on that same model. Holding the model fixed is what makes the difference attributable to the descriptor rather than to the model. It is one extra pass over your evaluation set, about **56 runs and US\$0.27** at the cheap tier. See the run table in D4.

D2(c) · Calling more than one tool in a turn — the cost lever

Class 4's loop parses **one** Action: per turn. Real tool-calling APIs (Application Programming Interfaces) do not work that way: a model can return several tool calls in one response, and a well-built loop executes them together and appends all the observations before asking again.

This matters because turns are the expensive axis. From **Class 4 Capsule 3**, which derived it, and **Class 5**, which put it in code: the loop is stateless, so the whole trajectory is re-sent every turn. For T turns, a base prompt B and D tokens added per turn:

$\text{input} \sim B \cdot T + D \cdot T^2/2$	Class 4 Capsule 3, the form on the slide
$\text{input} = B \cdot T + D \cdot T(T-1)/2$	Class 5's notebook, the exact sum

Both are the same model — the first drops the -1 because it makes no difference to the argument. At $T = 12$, $B = 1,200$, $D = 400$ they give 43,200 and 40,800. **Use either, say which.** The prefix B is re-sent every turn, and the accumulated history grows quadratically in the turn count T. Cutting turns cuts both terms — the second one hard, and “*doubling the turns from eight to sixteen does not double the bill; it nearly triples it.*”

What A2 requires.

1. **Extend the loop** so it can parse and execute a set of tool calls in one turn. Concretely: parse an Action: block rather than a single Action: line, execute each, append each observation. *The starter scaffold (Wed 2 Sep) ships this.*
2. **Write down the dependency rule** — which of your tools may be called together and which may not. A pair can go in parallel only when **neither needs the other's output**.
3. **Measure it.** Run the same evaluation set sequentially and in parallel, and report turns, tokens and cost for both.
4. **Show correctness did not move.** Same pass rate, or explain the difference.

Worked example — Problem A, and why this is worth doing. Turn 1 must run alone: `get_claim(claim_id)` — everything else needs the member, hospital and **line items** it returns. But once you have it, the checks on **each line** depend on nothing but that record, and neither do the policy and hospital lookups. They can all go together:

```
Turn 1:  get_claim("CLM-8842")           -> 3 line items
Turn 2:  lookup_policy(member_id)
        || check_coverage(47120) || check_coverage(31255) || check_coverage(62480)
        || get_hospital_status(hospital_id)
Turn 3:  get_preauthorisation(member_id, 62480)  <- only this line needed one
Turn 4:  issue_decision_letter(...)             <- behind the gate
```

Notice what parallelising does and does not collapse. The three coverage checks fold into one turn because they are independent of each other. **The pre-authorisation cannot** — you do not know which line needs one until coverage has answered. That is your dependency rule, and a claim with no pre-authorisation required finishes a whole turn earlier.

Count the calls, because the turn count follows from them. There are **eight tool calls** here — `get_claim`, `lookup_policy`, `check_coverage` three times, `get_hospital_status`, `get_preauthorisation`, `issue_decision_letter`. Run one per turn and that is **eight turns**. Fold the five independent ones into turn 2 and it is **four** — which is exactly the “turns”: 4 on the CLM-8842 record in Appendix A. **The gated action is a turn like any other**; it is gated, not free.

With $B = 1,200$ and the same total observation volume (eight observations of about 400 tokens, so D doubles when the turns halve):

	Turns	Input tokens $\approx B \cdot T + D \cdot T(T-1)/2$
Sequential	8 ($D = 400$)	$9,600 + 11,200 = \mathbf{20,800}$
Parallel	4 ($D = 800$)	$4,800 + 4,800 = \mathbf{9,600}$

The same work, the same observations, 54% fewer input tokens — because the history was re-sent four times instead of eight. Nothing was cut; the calls were only regrouped. Notice **where** the saving comes from: the prefix term fell by half, but the quadratic term fell by more than half — from 11,200 to 4,800. **That is the T^2 behaving as advertised**, and it is why cutting turns is the biggest lever you have.

Worked example — Problem B, where the same technique pays less. Take REF-5602, the booking in Appendix A. Six calls, and a **deeper dependency chain**: the urgency band comes out of the criteria check, the band sets the window, the window decides which slots you may query, and only then can you book.

```
Turn 1:  get_referral("REF-5602")
Turn 2:  check_referral_criteria(OPH, referral)  || lookup_patient(P-1180)
Turn 3:  get_clinic_slots(OPH, 09-09..09-30)
        || get_clinic_slots(OPH, 10-01..11-04)    <- both windows at once
Turn 4:  book_slot(...)                          <- behind the gate
```

Six sequential turns become four, and on the same assumptions that is **13,200 → 8,400 input tokens, a 36% saving — not Problem A's 54%**. The reason is structural, and it is the lesson: **parallelising can only collapse calls that are independent of each other**. Problem A had five such calls and folded them into one turn. Problem B has only two — the criteria check and the patient lookup — because band → slots → book is a chain, and a chain cannot be shortened by running things at once.

And look at turn 3 carefully, because it is a gamble. Firing both slot queries together is what makes this run four turns instead of five — but if the near window had contained a slot, the second query was wasted. **That is the parallel-versus-sequential trade in one line**, and REF-5602's slots_queried: 2 is the honest record of having taken it. Same grouping reconciles the other two records in Appendix A: REF-5590 escalates in three turns, REF-5614 asks in two.

THESE ARE EXAMPLES, NOT THE PRESCRIBED ANSWER

Both groupings above are **one defensible reading** of the dependencies, not the required one. A team that queries slots sequentially and reports five turns, or splits the coverage checks differently, or keeps a call alone because it wanted the observation before deciding what to do next, **has not made a mistake**. Where the parallel boundary falls is your design judgement, and D2(c) marks the reasoning, not the number.

What we do require is that you **state your dependency rule, measure both ways, and show correctness did not move**. A team that parallelised less than we did and explained why is on stronger ground than one that copied this page.

Two honest limits, and we expect you to find them. Parallel calls can *raise* cost when a call turns out to be unnecessary — sequentially, an early observation might have told you to skip it. And they remove a decision point the model would otherwise have used. Say where your dependency rule draws the line and why.

D3 · The guardrail layer, and the guardrail checklist

Two things, and they are different.

(a) The code layer, shipped before any prompt tuning: a **step cap**, a **budget ceiling**, **action de-duplication**, and an explicit **autonomy setting** — *suggest*, *confirm*, or *act* — with the gate placed in front of the irreversible step rather than in front of the agent as a whole. State which setting you chose and defend it in the report.

(b) The guardrail checklist — at least 10 test cases. These are not the same as evaluation cases. An evaluation case asks *did it get the job right?* A guardrail case asks *did it refuse, cap, or escalate when it should have?* Each of the ten must name the wrong behaviour it exists to catch, and state the observed result. Class 6 (Responsible AI, failure modes and the frontier) will give you the OWASP (Open Worldwide Application Security Project) LLM (Large Language Model) Top 10 for 2025 as a source of categories — but the code-layer material you need is already in Class 4, so **do not wait for Class 6 to start this**.

At least **three** of your ten must cover the request text itself being hostile — both problems contain free text written by someone outside your organisation: a member's claim narrative, and a referring doctor's clinical summary.

Run the checklist on the scripted backend. A step cap, a budget ceiling, action de-duplication and the autonomy gate are *your code* — a model cannot influence whether they fire. So a deterministic backend is not merely the cheap way to test them, it is the correct one: a marker re-runs your checklist and gets your table back. **These ten cases cost nothing and need no key.**

One caveat, on the three hostile-text cases. A scripted run proves **your guardrail fires** when the agent attempts the bad action — you script the attempt. It cannot tell you whether a live model is *talked into* attempting it in the first place. Those are two different questions, and D3(b) asks the first. If the second interests you, it is an observation from your D5 battery, not a guardrail case.

D4 · The evaluation set — 30 to 50 cases

Both problems have exactly three outcomes: one that ACTS, one that ASKS for something named, and one that ESCALATES. Everything in this deliverable is written in those terms, so it reads the same whichever problem you chose.

- **30–50 cases**, written by the team. Across a team of 6 or 7 that is 5–8 each.
- **Outcome-graded** — grade what the agent concluded, not the path it took there.
- **Negative cases** — **2 is the floor, 6 to 10 is what a set this size should carry.** A negative case is one whose correct outcome is **the ask or the escalate — anything except the act.** Each must name the wrong behaviour it exists to catch. A negative case that actually **fired** during your development, and changed something, earns explicit credit.
- **Isolation** — each case starts from a clean state; no case may depend on a previous one having run.
- **Two kinds of check, and you need both** — see *how your harness decides a case passed*, below.
- **A measured pass rate** — **one per model for your final prompt version**, plus the **v1-versus-v2 pair on a single model** for D2(b). Always with the trial count. (*Prompt version* means your v1 and v2 from D2(b). We avoid the word *policy* here: in Problem A it already means the member's insurance policy.)

How many runs — the arithmetic, so nobody has to guess. A **trial** is one run of one case. Ordinary cases get one trial; **negative cases get three**, because they are the ones that flip between runs and a single trial cannot tell a real refusal from a lucky one. That is **two extra trials per negative case**, per model.

	Minimum that passes	The shape we expect
Cases in the set	30	40
...of which negative	6	8
Trials per ordinary case, per model	1	1
Trials per negative case, per model	3	3
Live models (D5b)	3	N – 1 — 5 in a team of 6
Runs per model	$30 + (6 \times 2) = \mathbf{42}$	$40 + (8 \times 2) = \mathbf{56}$
The battery — final version $\times$ models	$42 \times 3 = \mathbf{126}$	$56 \times 5 = \mathbf{280}$
+ v1 on one model, for D2(b)	42	56
Live runs, total	168	336
Live runs per member	42	56 — one battery each

Why N – 1, and why the last row is the one that matters. N is your team size. The *minus one* is the v1 pass: with D3(b), D5(a) and D7 all running on the scripted backend, the only live work in this assignment is the D5(b) battery and D2(b)'s v1 run. So a team of six has six live jobs and a team of seven has seven — **one full battery each, and nobody watches.** Adding models does not raise anyone's bill: section 7 prices *one member, one model*, and 56 runs on the cheap tier is US\$0.27 whether your team runs three models or six.

The v1 comparison does NOT double the battery, and this is a design rule, not a budget dodge. To compare two prompt versions you hold the model fixed; to compare models you hold the prompt fixed. **Change one thing at a time or the difference is not attributable to anything.** So D2(b)'s v1 runs once, on one model — the same model you will quote its v2 against — and the three-model battery runs your final version only. Running v1 on every model buys you nothing and multiplies the bill.

Those are LIVE runs only, and they are what section 7 prices. The scripted backend is free, deterministic and needs no key — run the whole set there as often as you like, and do all your debugging there. **One member owns one model (D5b)**, so **56 runs is what any one member carries** — never the whole battery.

Going above these numbers is allowed and going below is not. More trials on everything is a better experiment and a bigger bill; if you raise them, say so with the trial count beside every pass rate. **A pass rate reported without its trial count is not a measurement.**

How your harness decides a case passed. There are only two ways to grade an answer, and your set needs both. **The names do not matter; the difference does.**

	A CODE CHECK	A JUDGEMENT CHECK
What it is	Your harness compares the agent's answer with your answer key. <code>decision == "escalate"</code> . No model, no person, no opinion.	Someone reads the record and decides whether it is good enough. That someone is either a person on your team or a second model you prompt to grade — that is all “LLM-as-judge” means.
Use it for	The decision. The single trigger. The named missing item. The clinic, date and time. Whether the gated action fired exactly once.	Whether the stated reason is actually a reason. Whether the evidence trail supports the decision. Anything where two correct answers can be worded differently.
It costs you	Nothing, and it gives the same result every time you run it.	A person's time, or tokens you pay for.

The split, in one line: the decision and the trigger are code checks; the wording is a judgement check. Most of your set should be code-checked, because most of what you are grading is a value from a fixed list of three. A handful need judgement. **Your results table must say which check each case used.**

	Code check — assert a value	Judgement check — read it and decide
Problem A	The key says <code>decision = escalate</code> , <code>trigger = annual_limit_exceeded</code> . Assert those two fields match. Nothing else.	Does the reason actually name the claim total and the amount remaining — or does it only say “cannot be decided at this level”?
Problem B	The key says <code>decision = book at OPH-C2</code> , <code>2026-10-14</code> , <code>11:20</code> . Assert clinic, date and time.	The record claims a routine band, an eight-week window and a booking at five weeks. Does that justify the date, or was it simply the first slot returned?

Where the right answer comes from. For the records we shipped, it is in `expected_outcomes_A.json / _B.json` — join on `case_id` and compare. **For the cases you invent, you write the labels**, in the same shape, and the quality of those labels is part of what D4 is marked on.

If a model is your judge, it is a measuring instrument. Name it, keep the grading prompt in the repository, and use a different model from the one being graded. An undocumented judge is not a measurement.

And check the thing you care about, not a string that usually accompanies it. A substring check can pass for the wrong reason — Class 4 showed you one that did.

Why 30–50 and not 10. A 10-case set cannot tell two prompt versions apart. Class 4's demonstration set separated a careful agent from a plausible-looking one only because the negative cases were in it — and only 6 of the 8 cases were ordinary. Size is what makes the number mean something.

D5 · The model battery — one scripted run, one live model per member

Two runs of the same harness, and this is the part that makes the harness an instrument rather than a form.

(a) The scripted run. Your submitted harness must run **end to end on a scripted backend** — deterministic canned responses, no network, no key — so that a marker can clone the repository and reproduce your numbers at zero cost. `BACKEND = "scripted"` must be the default in the submitted code. **If it does not run this way, Technical Execution is capped.**

(b) The live battery. Run the same evaluation set against **at least three different live models** through OpenRouter and report **relative** performance — not which model is “best” in the abstract, but which model does *your* job at what cost, and where they diverge. Expect them to diverge most on the **negative cases**; that is the finding to look for.

Your code must be **vendor-neutral**: one `BACKEND / MODEL / BASE_URL` block at the top and exactly one function that knows a vendor exists. Switching model is a string. This is the Class 4 pattern; copy it.

Sizing the battery so it fits the budget — see section 7. The run counts are in D4: one trial per ordinary case, three per negative case. Split the models across team members — **one member owns one model, and runs it on their own key.**

How many models: three is the floor, N – 1 is the shape we expect. N is your team size. Three models is what passes; **five in a team of six, six in a team of seven** is what we are asking for, because the remaining member owns D2(b)'s v1 pass and that accounts for the *minus one*. Every member then runs one full live battery on their own key. **It costs no one anything extra** — 56 runs is 56 runs whether your team fields three models or six.

Two conditions, or the comparison is mush. (1) The models must actually differ: your set must span **at least two price tiers**, and no two members may take models from the same family. Six mid-tier models from two vendors is not a comparison. (2) **The evaluation set and the prompt must be identical across every member** — same commit, same v2 prompt, MODEL the only string that differs. With three runners drift is survivable; with six it silently voids the whole battery.

Reporting five or six models inside a 350-word section. Section 3 of the report is 350 words and that is not changing. **Tables do not count toward the word cap**, so put *every* model in the results table and spend the prose on **the spread and the two extremes** — the cheapest model that met your bar, the most expensive one that did not earn its price, and where the negative cases separated them. A tour through six models at one sentence each is worse writing than three discussed properly.

Who runs what — the live measurement only, for a team of six. This table is not a division of the assignment. It divides **the last step**: one member, one model, one key, numbers with their name on them.

Member	Owns	Prompt version	Live runs
1	Live model A — cheap tier	v2 (final)	56
2	Live model B — cheap or mid tier	v2 (final)	56
3	Live model C — mid tier	v2 (final)	56
4	Live model D — a different family	v2 (final)	56
5	Live model E — the outlier you want to test	v2 (final)	56
6	The v1 pass — on a model already in the battery, for D2(b)	v1	56
7 (if you are seven)	Live model F	v2 (final)	56

Member 6's run is not a lesser job. The v1 pass is the only measurement in this assignment that isolates *your own writing* as the variable — it is what makes D2(b) a finding rather than an assertion. It must sit on a model someone else already ran, because comparing prompt versions means holding the model fixed.

Everything else — how you divide it is your team's call, and we are not going to prescribe it. The agent, the tools, the guardrail layer, the harness, D5(a)'s scripted run, D3(b)'s ten cases and D7's two failures are **the build**. That is two weeks of real engineering, it is where most of the marks are, and it is not a side-job hung off the table above. Divide it however suits your team's skills, and **name the owners in section 4 of your TEAM_DECLARATION**. The strands listed there and in the FAQ are a suggestion you are free to discard.

What we do fix is the ending. Whatever you agree about the build, when the battery is run every member runs one — because a member who never sends a live request has not met a real model's latency, variance or refusals, and that is a thing this course is supposed to leave you with.

D6 · The cost-to-serve model — three layers, and the four levers you pulled

Use the Class 5 three-layer model. You built it in the Class 5 cost-to-serve notebook; reuse that notebook rather than starting again. Use your **measured** success rate from D4 and your **measured** token counts from D5 — not estimates.

Layer	What it is	As volume grows
1 · Per-task variable	Input tokens + output tokens + any retrieval or tool fees.	Linear. Never amortises.
2 · Per-task expected fallback	(1 – success rate) × cost of handling one failure.	Linear. Never amortises. Usually the largest layer, and the one almost every cost model omits.

Layer	What it is	As volume grows
3 · Fixed monthly	Storage, infrastructure, your eval runs, monitoring, maintenance.	Gets cheaper per task forever.

```

input tokens      ~ B*T + D*T^2/2      B = base prefix, D = growth/turn, T = turns
layer 1 variable  = input*price_in + output*price_out + retrieval + tool fees
layer 2 fallback  = (1 - success_rate) * failure_cost
cost per successful task = layer 1 + layer 2
monthly          = cost per successful task * volume + layer 3

```

Build your baseline on exactly this. Plain input and output tokens at your model's list price. Do not put anything cleverer in the baseline — the two adjustments below are real, but they are model-specific and they will make your headline number impossible to check.

Price layer 2 from labour, not from a guess. A failure is not free just because no human is named in the design. $\text{failure_cost} = \text{hourly rate} \times \text{minutes per escalation} \div 60$. Each problem statement in Appendix A gives you a default role and handling time; use it, or substitute your own and say why.

Two things move this number, and neither belongs in the baseline

Report your baseline first. Then, if either of these applies to you, report the adjusted figure beside it and say what changed.

(a) Prompt caching pushes the bill DOWN — sometimes a long way.

Your prefix **B** is the same text on every turn, which is exactly what caching exists for: the provider stores it and charges a fraction of the input price to re-read it. On the quadratic that is a real saving, because $B \times T$ is the term caching attacks.

But **the mechanism, the discount and the billing differ by vendor and by model**, and they change. Some providers cache automatically; some require you to mark the block; some charge a premium to *write* the cache before you save anything on reading it; some expire it in minutes. There is also a minimum cacheable size, and **a cache only hits on an unchanged prefix** — put a timestamp at the top of your system prompt and you have turned caching off without noticing.

So: do not model caching. Measure it. If you use it, run the same evaluation set with and without, take the token counts from the API response rather than from the price list, and report both. **A team that says “caching cut our input bill by X%, measured, on this model” has done something better than a team that assumed a discount.**

(b) A reasoning model pushes the bill UP — and it is easy not to notice.

Reasoning models produce hidden thinking before they answer. **Those thinking tokens are billed as output tokens**, and output is the expensive side: on the cheap tier in section 7 output costs **4×** the input price, on the frontier tier **5×**. A model that quietly emits 900 thinking tokens where a plain model emitted 100 has not made your agent a little more expensive; it has changed the shape of your bill.

How to see it: read the usage block the API returns, not your own estimate. Reasoning tokens appear in the output count. If output per turn jumps by an order of magnitude, that is what happened.

How to cap it, through OpenRouter. A reasoning object on the request, and which field applies depends on the model family:

```

"reasoning": {"max_tokens": 1024} // Anthropic, Gemini, some Qwen - a hard ceiling
"reasoning": {"effort": "low"}    // OpenAI o-series and GPT-5, Grok - ~20%
"reasoning": {"enabled": false}  // off, where the model allows it

```

effort is proportional rather than absolute — roughly *minimal* 10%, *low* 20%, *medium* 50%, *high* 80%. Anthropic's max_tokens has a floor of 1,024.

THE TRAP, AND IT CATCHES PEOPLE

reasoning: exclude = true stops the reasoning being *returned* to you. **The model still does it, and you are still billed for it.** Excluding is a tidiness setting, not a cost control. If you want to spend less, cap it or turn it off.

Our recommendation for A2: don't. A reasoning model is not what this assignment rewards — the marks are in the harness, and reasoning tokens will eat a US\$10 key quickly. If you do use one, **cap it, say what you capped it to, and report the cost**

both ways. That comparison is itself a good finding.

A sensitivity table, not a point estimate. Your success rate is an estimate and so is your failure cost. Show cost per successful task across success rate ± 10 percentage points, and say whether your conclusion survives the whole range. *A robust answer and a knife-edge answer deserve different amounts of confidence, and a sponsor is entitled to know which one you are handing them.*

A break-even success rate — the question that actually decides which model you ship.

Your model battery in D5 gave you a measured success rate for each model. The cheap model costs less per run and gets more wrong; every wrong answer costs you an escalation. **So the real question is not “which model is better” — it is:**

How good would the cheap model have to be before the money it saves per run stops being eaten by the failures it causes?

Three quantities, all of which you already have:

	What it is	Where it comes from
C	What one run costs on the cheap model — tokens only	your token counts $\times$ the price
E	What one successful task costs on the expensive model — its tokens plus its own failures	layer 1 + layer 2 for that model
F	What one failure costs — the escalation	your problem's figure in Appendix A

The logic in one line. Switching to the cheap model saves you $E - C$ on every task. One failure costs F . So the number of failures you can afford before the saving is gone is $(E - C) \div F$ — and the success rate you need is whatever is left:

$$\begin{aligned} \text{failures you can afford} &= (E - C) / F \\ \text{break-even success rate} &= 1 - (E - C) / F \end{aligned}$$

Why E includes its failures and C does not. For the expensive model you *measured* its success rate, so you can price its failures and fold them in. For the cheap model its success rate is the unknown you are solving for — so it cannot appear on that side of the sum. That asymmetry is the whole reason the formula looks lopsided.

Worked, on Problem A. Take a cheap-tier run at **US\$0.005** and a mid-tier run at **US\$0.049**. The mid-tier model measured **92%** on your set, and a failed claim costs a claims assessor **US\$7.60**.

$E = 0.049 + (1 - 0.92) \times 7.60 = 0.657$	the expensive option, all in
$E - C = 0.657 - 0.005 = 0.652$	saved on every task by going cheap
$0.652 / 7.60 = 0.086$	so you can afford ~8.6% failures
$\text{break-even } p = 1 - 0.086 = 0.914$	the cheap model needs 91.4%

Now read what that means, because it is the point of the exercise. The cheap model is roughly **ten times cheaper per run** — and it still has to land within about **one percentage point** of the expensive one to be worth choosing. **When failures are expensive, the price of the model barely matters. Accuracy does.**

And the reverse holds. If a failure cost **US\$0.76** instead of US\$7.60, the same arithmetic gives a break-even of **86.2%** — the cheap model could afford roughly 14% failures instead of 9%. **The cheaper it is to be wrong, the more the token price matters.** That is why Appendix A gives each problem a different failure cost, and why the same agent can be a good decision on one problem and a bad one on another.

Then answer it for your own system: does your cheap model clear its break-even, or how far short does it fall? **One sentence, with your two measured numbers in it, is the most useful thing in your report.**

Why this is not the “ $\div$ success rate” formula from Class 4 Capsule 3. Both are correct models of different worlds. Dividing by the success rate prices **retry until it works** — a failure costs you another attempt. Adding $(1 - p) \times \text{failure_cost}$ prices **escalate on failure** — a failure costs you a human. In both A2 problems a wrong outcome goes to a person, not back into the loop, so the escalation form is the right one. Say in one line which world your system is in; a team that uses the retry form and **justifies** it is not wrong.

The cost ledger — this is the part we mark hardest. A cost model that only reports a number tells us nothing about your design. Report the four levers you pulled, with a measured before and after for each.

Lever	What it attacks	Where you built it	Report
1 · Tool block size	B — re-sent every turn, called or not. Linear in turns.	D2(a) — the tools you cut	tokens of tool definitions, before and after your cuts
2 · Turn count T	The quadratic term. The biggest lever you have.	D2(c) — parallel calls	turns and input tokens, sequential vs parallel
3 · Observation size D	Compounds — every observation is re-sent on every later turn.	D2(b) — the descriptor rewrite	tokens returned per call, v1 vs v2
4 · Success rate	Sets layer 2 , usually the biggest layer. A cheap agent that is wrong half the time is not cheap.	D4 — the eval set	measured pass rate, and the cost per successful task it implies

Levers 1 and 3 look similar and are not: **a fat tool block is linear in turns, a fat observation compounds**. Both are worth cutting; only one of them explodes. Say which dominated your bill and how you know.

Then state the three caps that ship with it: **step cap, budget ceiling, monthly limit per user**.

This is marked on method, not on arithmetic precision. A defensible model with stated assumptions, three separated layers, a sensitivity range and four measured levers beats a confident single number with none of them.

D7 · Two reproduced failures — one of them a loop failure

Reproduce **two** distinct failure modes, and fix each **in the layer where it belongs** — code, tool interface, or prompt. Each must be built as a **deletion from your working agent**, not as a separately written bad agent: *“the working agent, minus X.”* Putting X back must recover the behaviour.

Failure 1 · A loop-control failure — this one is required

The agent goes round and round: it repeats an action it already took, or re-reads what it already knows, or never reaches a conclusion. Class 4 built exactly this — careful's guard chain **with one guard deleted** — and the result is the thing worth understanding:

8 turns, no answer, 1.6× the cost, and no exception was raised. It did not crash. Nothing failed. It just burned money in a circle — and a loop has no memory of its own actions unless you give it one. **You only ever see this if you are counting.**

Four things to report:

1. **The instrumentation that found it.** Turns and cost logged per run. You cannot report a failure you had no way of noticing.
2. **The turn distribution across your whole evaluation set** — median, worst case, and how many runs hit your step cap. One number is not a distribution.
3. **The fix, in the code layer** — action de-duplication, a step cap, a budget ceiling. Say which one actually caught it and why the other two would not have.
4. **Before and after: turns, tokens, cost, and pass rate.** A step cap that stops a runaway also truncates a legitimate long run — show that your pass rate did not fall.

Set your caps from evidence, not from a round number. If your median run is 4 turns and your worst legitimate run is 7, a step cap of 8 is defensible and a step cap of 30 is decoration. Say where yours came from. **And make the stop loud.** A cap that silently returns an empty answer is worse than the loop — it converts a visible cost problem into an invisible correctness problem.

Failure 2 · A different layer

Your choice, but it must sit in the **tool interface or the prompt**, not in loop control again. Class 4's worked example was a fat, stale observation carrying a landmine — 246 tokens that produced a confident wrong answer, fixed at the interface rather than by adding a sentence to the prompt.

For both failures, state **which layer the fix belongs in and why the other two were the wrong place**. That judgement is most of the mark.

Both failures run on the scripted backend, and so do their before/after tables. A failure built as a deletion from your working agent is deterministic by construction — that is the whole point of building it that way. Script the observation that induces the loop, and the loop is reproducible for ever, at zero cost. **D7 does not need a key and must reproduce for a marker.** The turns, tokens and cost you report are the instrumented counts from those scripted runs; the only live numbers in this assignment come from D5(b).

4 · WHAT YOU SUBMIT

Four artefacts, in one archive named PE6201_A2_[TeamID].zip — your team identifier as issued at the start of the course, for example PE6201_A2_B-4.zip. Plus the video link.

#	Artefact	Notes
1	Code repository — in TWO places	(a) A public repository your team creates on GitHub or equivalent, and (b) a copy of the same code files inside your NTULearn submission folder . Both are required: the repository shows history and contribution, the folder copy means marking never depends on a link working. Include a README that gets a stranger from clone to a reproduced scripted run. Contains the agent, tools, guardrail layer, evaluation set, harness, fixtures, result tables and the cost model. All of the numbers live here.
2	Team report — at most 2,000 words	Argument, not data. It follows the structure below and it starts with why an agent. Tables and figures do not count toward the word cap; prose does.
3	Recorded demonstration — 5 minutes	The system running, one negative case shown live, and the numbers. Every member speaks. Over-length is penalised under Communication.
4	Team self-appraisal	One collective sheet per team — not one per member. Completed together, rating the team against Rubric 1, with a short reflection on your trade-offs. Checked into your A2 submission folder alongside everything else. Ungraded, but required — a missing self-appraisal is an incomplete submission.

Contribution log. The repository must contain a CONTRIBUTIONS.md naming who did what, and the commit history must corroborate it. This is what section 8 relies on.

The report — required structure, with a word budget

Six sections, in this order. The per-section budget is guidance; the **2,000-word total is a limit**. Numbers, tables and figures live in the repository and do not count toward it.

#	Section	What goes in it	Words
1	Why an agent	D0 — the rung you are on and what rungs 1 to 6 would and would not have delivered; the two tests, including your own $S = P^{(1/T)}$ figure; what good looks like.	400
2	The tool layer	D2 — why this set and what you cut; what your descriptor rewrite measured; your dependency rule and what parallel calling saved.	450
3	What the evidence showed	D4 + D5 — pass rates, where the models diverged, what the negative cases caught.	350
4	What it costs	D6 — the four levers and which dominated, the monthly figure at your problem's volume, your sensitivity range and break-even. Note caching or a reasoning model here if either applies.	400
5	The two failures	D7 — the loop failure, how you detected it, the fix and its before/after; the second failure and why its layer.	250
6	What we would not deploy	The limits you found, plus a paragraph on an architecture you did not build — what it would have caught, what it would have cost, why you stayed single-agent.	150

The budget is shaped by where the marks are, and by which arguments are hardest to make briefly. Sections 1 and 2 together are 850 words — nearly half — because Conceptual Understanding (25%) and Reasoning & Justification (25%) are

half the rubric, and almost all of that lives in *why this rung* and *why this tool set*. Section 4 grew because a cost model now has four measured levers and a range to defend, not one number to state. Section 5 is short on purpose: two failures, tightly described, is more convincing than two narrated at length.

Section 1 is read first and read hardest. A report that opens with what the team built, rather than why an agent was the right instrument, has skipped the question the whole of Class 4 was arranged around.

5 · HOW A2 IS MARKED

Rubric 1, four criteria, applied to the team submission. What each criterion looks for in A2:

Criterion	What earns the top band here
Conceptual Understanding — 25%	You can place the problem on the ladder and defend the rung — what a single prompt, a fixed workflow or read-only agentic retrieval would and would not have delivered, and what the climb cost. The pre-build diagnostic names a system of record that can contradict the model at machine speed. You can say where a ReAct loop stops being the right architecture , citing evidence rather than taste. The distinction between an evaluation case and a guardrail case is held throughout. The autonomy setting is chosen against the irreversible step, not by taste.
Technical Execution — 30%	The scripted run reproduces. Turns and cost are instrumented per run, so the loop failure is something you detected rather than assumed. The gated action logs a decision with its evidence trail and its gate — effort spent on documents, interfaces or integrations earns nothing here. The loop genuinely executes several tool calls in one turn. Every tool carries a full six-field descriptor, with real poka-yoke moves in the signatures. The evaluation set is isolated, outcome-graded, multi-trial and contains real negative cases. The guardrails are in code, not in the prompt. The live battery ran and the numbers are the ones the code produced.
Reasoning & Justification — 25%	An argued paragraph on an architecture you did not build — what it would have caught, what it would have cost, why you stayed single-agent — with evidence behind it. The tool list is the shortest defensible one, scored against the three questions, with at least one tool removed or one “we tried not adding it” documented. The descriptor rewrite is measured, and reported honestly even if it did not help. The cost ledger names all four levers and says which dominated. The model choice follows from your own evidence. The two failures are located in the right layer, with the wrong layers named.
Communication & Clarity — 20%	The report argues rather than narrates. The demonstration shows the negative case, not only the happy path. The repository is navigable by someone who was not in your team. Limits are stated plainly.

Where the weight of the work sits. Within Technical Execution and Reasoning, the harness, the guardrails and the evidence carry the marks. **An agent that does more, proved less, scores lower than an agent that does less, proved properly.**

6 · USING AI TO BUILD AI

Encouraged, on four conditions, unchanged from A1:

1. **You must be able to explain every block of code you submit — what it does and why it is there. Block, not line: you are not being tested on syntax.** For A2 this applies team-wide: any member may be asked about any block.
2. Figures in your report are **measurements you ran**, not numbers a model produced for you.
3. Sources and tools used are attributed.
4. **Nothing is submitted twice.** A2 work does not appear in anyone's End-of-Course Project, and no Project work appears in A2 (section 1).

7 · BUDGET — THE US\$10 KEY

Each member holds a personal OpenRouter API key capped at **US\$10 for the whole course, with no top-ups**. A2's live battery must fit inside what remains after A1, alongside the End-of-Course Project.

One agent run of ~12 turns sends roughly **43,200 input tokens and 1,200 output tokens** — the Class 5 arithmetic, on the Class 5 price table. Prices are US dollars per million tokens, checked against vendor pages on 28 August 2026; they are the same numbers as the Class 5 cost-to-serve notebook, so you can reuse that notebook to re-price your own battery.

Tier	Price (in / out)	One run	40 cases × 1 trial	+ 8 negatives × 2 extra	56 runs — one model, one member
Cheap	0.10 / 0.40	≈ US\$0.005	≈ US\$0.19	≈ US\$0.08	≈ US\$0.27
Mid	1.00 / 5.00	≈ US\$0.049	≈ US\$1.97	≈ US\$0.79	≈ US\$2.76
Frontier	5.00 / 25.00	≈ US\$0.246	≈ US\$9.84	≈ US\$3.94	≈ US\$13.78

Read the last row again. One member running one full battery on a frontier model spends more than the entire course allowance. This is not a trap we set — it is the actual shape of the decision the course has been teaching since Week 3, and it is why the cheap tier is where an eval battery belongs. If you want a frontier model in your comparison, run it on the **negative cases only** and say in the report that that is what you did.

One member pays a little more. D2(b)'s v1-versus-v2 comparison is one extra pass over the set, on the cheap tier only — another **56 runs, about US\$0.27**. Whoever owns the cheap model carries **112 runs, about US\$0.54**. Everyone else runs 56. **The battery itself is your final prompt version on every model; v1 runs on one model only** — see D4.

Four rules that keep you inside the budget.

- **Debug on the scripted backend.** Live tokens are for the final battery, not for finding your bugs.
- **Only D5(b) is live. D3(b), D5(a) and D7 all run scripted** — the guardrail checklist, the reproducible end-to-end run, and both reproduced failures with their before/after tables. None of them needs a key, and none of them is priced above. If you find yourself spending live tokens on any of the three, stop: you are measuring your own code with an instrument that cannot see it.
- **One member owns one model, and every member owns one.** Three models is the floor; **N – 1 models for a team of N** is the shape we expect, the remaining member taking D2(b)'s v1 pass. Five models and a v1 pass in a team of six. This costs no one anything extra — the table above prices *one member, one model*, so US\$0.27 on the cheap tier is US\$0.27 whether your team fields three models or six.
- **If your estimated live spend exceeds US\$3 per member, your battery is too large** — cut trials or cases, or move a model down a tier, and say in the report that you did, and why. Budgeting against a fixed allowance is part of what this course teaches; it is not a restriction we apologise for.

Running out is a diagnosable event. Email early and you get help finding where it went and a plan to finish with what remains — not more credit.

8 · TEAMS, CONTRIBUTION, AND THE FREE-RIDER CLAUSE

Your team is already set. A2 is done in the group you were placed in at the start of the course — A-1 to A-9, B-1 to B-9, C-1 to C-10. **Teams are not re-formed, swapped or merged for A2.** If you are unsure which team you are in, ask before the check-in date rather than after. Cross-section teams do not exist and cannot be created — the check-in and the peer rating both run per section.

Team declaration — due Friday 4 September 2026, 23:59 SGT

Not an email. Check a file into your A2 submission folder, named TEAM_DECLARATION — .docx, .pdf or .md, whichever suits you. **A template is on NTU Learn from release (TEAM_DECLARATION.docx); fill it in and check it in.** If you would rather write your own, it must contain:

1. **Team ID and section** — for example B-4, Section B.
2. **Every member's name and matriculation number.**
3. **The problem you have chosen** — A or B.
4. **Who owns what** — the strands from the FAQ, or your own split. Names against work.
5. **Your repository URL.**
6. **The contribution statement — and this one is not optional.**

ON POINT 6, PLAINLY

State one of two things. Either **“All members are contributing.”** — say it explicitly; do not leave the section blank. Or **name the problem**: if a member has not engaged, has not responded, or is not delivering what was agreed, **say so here, by name, with what was agreed and what has happened.**

Why this matters more than it looks. The checkpoint exists so a crack surfaces while there is still time to fix it. When a team names a non-contributor on 4 September, the instructor issues a **written warning** and the member has nine days to remediate — which is usually enough, and most of these resolve. A team that stays silent on 4 September and raises the same problem on 13 September has a much weaker case: there was no warning, no opportunity to remediate, and no contemporaneous record. **Silence at the checkpoint is read as “all members are contributing.”**

Naming a problem here is not an accusation and it does not by itself cost anyone marks. It starts a conversation with a deadline attached.

This declaration is mandatory. A team that files nothing has an incomplete submission.

Marking. The instructor marks the team submission, once, against Rubric 1. Every member receives that team score by default.

Peer rating — opens 13 September, due Wednesday 16 September 2026, 23:59 SGT. After you submit, each member rates teammates' contribution through NTU's Group Project peer-review function (self-ratings excluded). **This does not scale anyone's mark.** It is a signal only: if it flags a member as a low or non-contributor, the instructor reviews the evidence and adjusts that member's mark alone, leaving the rest of the team on the team score. **Completing it on time is a participation requirement.**

Non-contribution. A documented non-contributor receives **0 for A2** instead of the team score. This is applied only when corroborated by **two independent signals** — a majority of teammates flagging the member through the peer-rating tool **and** no traceable work in the commit history or contribution log — and confirmed by the instructor. **Peer opinion alone is never sufficient.** Where the team raised it at the 4 September checkpoint, the written warning and the remediation window come first. The student may respond, and the standard single regrade request applies.

9 · WHAT FEEDBACK YOU GET, AND WHEN

Stated plainly so nobody is waiting for something that is not coming. **Feedback is issued at the instructor's discretion as to timing** — chasing it on a particular date is not a good use of anyone's week.

On	What you get	When
A1	Your mark, and written comments against the rubric through NTULearn. Comments concentrate on one rubric criterion, and on any place where your self-appraisal and the marking diverge sharply.	Timing at the instructor's discretion
A1 — cohort-level analysis	Not being run this trimester. There is no class-wide session on A1 mark distribution or common mistakes. A1 is due the same day A2 is released, and Class 5 falls the day after that, so there is no point in the calendar where such a session could honestly be prepared. This is a deliberate decision, taken so that individual feedback lands properly rather than a cohort session landing late and thin.	—
Your Project problem statement	Written feedback in the instructor-feedback box on the template.	Timing at the instructor's discretion
A2 (this assignment)	Feedback is shared offline after submission — Week 6 is the final session, so there is no class left to discuss it in.	Timing at the instructor's discretion

What this means for A2 in practice: do not plan to use A1 feedback as your guide to what a good A2 looks like. **Section 5 of this brief is that guide** — it says, criterion by criterion, what the top band looks like for this assignment. Read it before you start, not after you finish.

10 · TIMELINE

Date	What
Sun 30 Aug 2026	A2 released. Choose your problem, then write D0 before you write code — the ladder, the diagnostic, what good looks like. Then D1, D2 and the code layer of D3. All of it is Class 4 material you already have.
Mon 31 Aug / Tue 1 Sep	Class 5 — cost-to-serve and ROI (Return on Investment). Feeds D6 . Bring your problem's volume figure.
Wed 2 Sep 2026, midday	Fixture-data generator and starter scaffold notebook on NTULearn.
Fri 4 Sep 2026, 23:59 SGT	Team declaration due — a file in your submission folder, not an email (section 8). It must state whether every member is contributing. By now D4's evaluation set should be written, even if it has not run.
Mon 7 Sep / Tue 8 Sep	Class 6 — responsible deployment, failure modes, OWASP LLM Top 10 (2025). Sharpens D3(b) ; the red-team build in class is directly reusable.
Sun 13 Sep 2026, 23:59 SGT	A2 due . Repository, code files in the folder, report, demo link, team self-appraisal.
Wed 16 Sep 2026, 23:59 SGT	Peer rating due — a participation requirement.
Sun 27 Sep 2026, 23:59 SGT	End-of-Course Project due — do not let A2 eat this.

Because Class 6 falls five days before the deadline, **build the guardrail layer from Class 4 material now and let Class 6 sharpen it**. A team that waits for Class 6 to begin D3 will not finish.

11 · WORDS THAT DO REAL WORK IN THIS BRIEF

Term	What it means here
The ladder	Class 4's seven rungs, cheapest first. Rungs 1–6 are workflows; rung 7 is an agent. Go down the list only when the rung above cannot do the job.
External tool	A function the model calls that reaches information the model does not carry in its weights. Your own data files count, and they are what we expect . A tool that reaches the public internet is optional — permitted, never required, and only if its responses are recorded as fixtures so the scripted run still reproduces. See the scope boundary in D1.
Routing rule	Which of the three outcomes applies, and when. Appendix A states it as a table, one per problem. It is the organisation's policy, not yours — you automate it, you do not redesign it. The answer key is written against it, so a team that changes it cannot be marked against anyone else.
Gated action	The one step that cannot be undone. Problem A: issuing the decision letter — once a member has been told “approved in principle”, walking it back is expensive. Problem B: booking the slot — it consumes a scarce appointment and tells a patient a date. The gate goes in front of that step, not in front of the whole agent . In code it is one function: check the gate, write one record to a file. It does not build a letter, a booking system, or an app.
Case	One task you give the agent, with a known correct outcome. Your evaluation set has 30–50 of them.
Run	One complete execution of the agent on one case, start to finish — request in, outcome out. The loop iterates several times inside a single run , and that iteration count is T . So one run = T turns, and a run is finished when the agent produces a final outcome or a cap stops it.
Trial	One run of one case. Several trials of the same case, because the answer is not always the same. D4 sets the counts: one trial per ordinary case, three per negative case — so 40 cases with 8 negatives is 56 runs per model.
Turn, also called a step	One pass around the loop — the model thinks, calls one or more tools, and reads back what they returned. In this brief the two words mean exactly the same thing ; Class 4 used “turn” for the loop and “step” in the reliability arithmetic, and both are kept here because both appear in the material you have. T in every formula in this brief is the number of turns in ONE run — never the number of runs, cases or trials. A step cap is a limit on turns within a run.
Pass rate (P)	Passing trials ÷ total trials, stated with the trial count and the model. A property of whole runs , not of individual turns.

Term	What it means here
Per-step reliability (s)	How often a single turn goes right. You never measure it directly — you back it out with $s = P^{(1/T)}$. A diagnostic, not a constant.
Evaluation case	A case with a known correct outcome. Asks: <i>did it get the job right?</i>
Guardrail case	A test of a refusal, a cap or an escalation. Asks: <i>did it decline when it should have?</i>
Negative case	An evaluation case whose correct outcome is the ask or the escalate — anything except the act . Both problems have exactly one acting outcome, so this reads the same for A and B.
Autonomy setting	<i>Suggest</i> (proposes, human does it) · <i>confirm</i> (proposes, human approves, agent does it) · <i>act</i> (agent does it).
Loop-control failure	The agent repeats an action, re-reads what it already knows, or never concludes. It does not crash — it burns turns. Visible only if you are counting them.
ReAct (Reason + Act)	The architecture A2 builds: one agent running one loop — thought, action, observation, repeat, final.
Multi-agent	Several agents that call or hand off to each other. Compared in Class 4 Capsule 1, never built, and out of scope for A2 — argue about it in the report instead.
ACI — Agent–Computer Interface	The tool names, signatures, descriptions and return shapes, taken together. It is the <i>entire</i> manual the model gets.
Descriptor contract	The six-field definition of one tool — name and signature, what, input, returns with a size bound, fails when, irreversible yes/no.
Poka-yoke move	A design change that makes a class of error impossible rather than discouraged — a closed <code>Literal</code> instead of a free string, an id instead of a name, a safe default.
Dependency rule	Which of your tools may be called in the same turn. A pair may go in parallel only when neither needs the other's output .
Cost-to-serve	Cost per successful task — layer 1 (per-task variable) + layer 2 (expected fallback). Not the same as cost per call.
Expected fallback (layer 2)	$(1 - \text{success rate}) \times \text{cost of handling one failure}$. Usually the biggest layer, and the one most cost models leave out.
Break-even success rate	The success rate a cheap model would need to match an expensive one, once failure cost is counted.
Scripted backend	Canned, deterministic responses, no network, no key. What a marker runs.

A fuller **FAQ** (Frequently Asked Questions) document accompanies this brief.

APPENDIX A

The two problem statements

Each gives you the situation, the three outcomes, the fixture data you must create, the minimum tool set, the gated action, the negative-case families, and the cost figures for D6.

Read both before choosing. They are matched in difficulty and both genuinely need a loop; they differ in what makes them hard — Problem A in the volume of checks a single claim can demand, Problem B in the cost of acting when you should not.

The tool sets are minimums, and the names are ours. `get_claim` and `lookup_policy` say what work has to happen. They are not an interface you must implement. **Rename them, change the arguments, merge two, split one, add your own.** D2(b) asks for your own six-field descriptor for every tool you ship, and you cannot write one for a signature somebody else fixed. Adding a tool needs a justification under D2(a); removing one needs evidence it never earned its place.

Two things are fixed: the routing rule and the gated action. The answer key is written against them.

PROBLEM A · Health-insurance claim first response

Situation. A regional health insurer receives claims from members after hospital treatment. **A claim is not one thing — it carries several line items**, each a separate procedure with its own code and amount, and each has to be checked in its own right. A claims officer opens the claim, checks the policy is live, works through the lines, chases a pre-authorisation where a procedure needed one, checks whether the hospital is on the panel, and then either approves in principle, writes back for a missing document, or escalates. Volume has doubled; they want the first response drafted automatically.

Three outcomes. Approve in principle · request a specific missing document · escalate to a human assessor.

Fixture data. members (member id, policy id, join date) · policies (policy id, product, status, start and end dates, annual limit, amount used to date, exclusions list) · claims (claim id, member id, hospital id, date of service, free-text narrative from the member, documents attached, **and a list of line items — each with a procedure code and an amount**) · hospitals (hospital id, panel or non-panel, country) · procedures (procedure code, description, requires pre-authorisation flag) · preauthorisations (member id, procedure code, reference, valid from and to).

Minimum tool set. `get_claim` · `lookup_policy` · `check_coverage(policy, procedure_code)` · `get_preauthorisation(member, procedure_code)` — *called only when coverage says one is required* · `get_hospital_status` · `issue_decision_letter` (the gated action — simulated).

Gated action. Issuing the decision letter to the member. Once a member has been told “approved in principle”, walking it back is expensive. **In code this is one log record** — the decision, the reason, the evidence trail, the gate. `issue_decision_letter` does not compose a letter; see the scope boundary in D1.

Negative-case families. Policy lapsed or outside its dates · **one line item excluded while the others are fine** (the claim is not simply approved or declined) · a procedure required pre-authorisation and none exists, or one exists but expired before the date of service · the line items together exceed the remaining annual limit · a duplicate of a claim already decided · **the member's free-text narrative instructing the system to approve.**

Which outcome, and when. This is the insurer's policy, not yours — you are automating it, you did not invent it, and you may not change it. It is also what makes your evaluation set gradeable: without a fixed rule, every team writes a different ground truth.

The situation	Outcome	What the record must carry
Every line resolves — covered, or covered once a valid pre-authorisation is found, or clearly excluded	approve in principle	A disposition for every line ; the approved total; and for each excluded line, the exclusion that caught it
A line needs pre-authorisation and none exists, or one exists but expired before the date of service	request document	The exact thing missing — “pre-authorisation reference for 62480, valid on 2026-09-02”. Never “more information”
A required document is absent — itemised bill, discharge summary	request document	Name the document, and the line it belongs to

The situation	Outcome	What the record must carry
Policy lapsed or outside its dates · lines together exceed the remaining annual limit · duplicate of a claim already decided · the narrative contains instructions aimed at the system	escalate	Who it goes to, and the single trigger that sent it there

A partly payable claim is still an ACT, not an escalation. Three lines approved and one excluded is *one decision letter* covering both. Escalate when the **claim** cannot be decided — not when a **line** is refused. Teams that collapse those two cases lose the distinction the assessor is actually paid for.

One successful run, end to end — and deliberately the awkward case. A three-line claim: two lines covered, one of those needing a pre-authorisation chased, and **a third line excluded**. The claim is still approved in principle, with the excluded line refused inside the same decision. Note that the agent checked three procedures and chased exactly one authorisation — a different claim would produce a different number of turns. This is the whole deliverable of the gated action; no letter was composed and nothing was sent.

```
{
  "ts": "2026-09-09T10:14:32",
  "case_id": "CLM-8842",
  "decision": "approve_in_principle",
  "reason": "Policy POL-3310 active to 2027-03-31. Hospital H-114 on panel.
    2 of 3 lines payable; 1 excluded. Approved total SGD 2,180
    against SGD 9,200 remaining on the annual limit.",
  "lines": [
    { "code": "47120", "amount": 1400, "status": "covered" },
    { "code": "62480", "amount": 780, "status": "covered",
      "preauth": "PA-5521 valid 2026-08-01..2026-10-31" },
    { "code": "31255", "amount": 300, "status": "not_covered",
      "exclusion": "EX-14 cosmetic dermatology" } ],
  "approved_total": 2180, "refused_total": 300,
  "evidence": [ "get_claim", "lookup_policy", "check_coverage x3",
    "get_preauthorisation", "get_hospital_status" ],
  "autonomy": "confirm",
  "gate": "operator approved at turn 4",
  "turns": 4,
  "tokens_in": 8940, "tokens_out": 410, "cost_usd": 0.0041
}
```

And the other two outcomes, in shape. Only the distinguishing fields are shown — the evidence trail, autonomy, gate, turns and cost are recorded on every outcome, not just on approvals.

```
// ASK - the missing thing is NAMED, and tied to the line that needs it.
// Note the ask still records what it DID resolve, including a refusal.
{"decision": "request_document",
 "case_id": "CLM-8888",
 "reason": "Line 62480 requires pre-authorisation. None found for member
           M-6118 covering the 2026-09-08 date of service.",
 "missing": {"item": "pre-authorisation reference",
             "for_line": "62480",
             "must_be_valid_on": "2026-09-08"},
 "lines_resolved": ["47120 covered",
                    "31255 not covered - EX-14 cosmetic dermatology"],
 "evidence": ["get_claim", "lookup_policy", "check_coverage x3",
              "get_preauthorisation"], "turns": 3}

// ESCALATE - ONE named trigger, and who it goes to
{"decision": "escalate",
 "case_id": "CLM-8925",
 "escalate_to": "human claims assessor",
 "trigger": "annual_limit_exceeded",
 "reason": "Claim total SGD 11,400 exceeds SGD 9,200 remaining on policy
           POL-3310. Lines were not individually priced: the claim cannot
           be decided at this level regardless of coverage.",
 "evidence": ["get_claim", "lookup_policy"], "turns": 2}
```

Read what the escalation did NOT do. It stopped after two turns — once the limit was breached, pricing the individual lines would have burned turns on a decision it was never going to make. **An early exit is a correct behaviour, not a truncated run,** and your evaluation set should contain a case that rewards it.

Why this problem needs a loop, and not a checklist. The number of steps is decided by the claim, not by you. **A claim with one line item and a live policy is a short run. A claim with four lines, two of which need a pre-authorisation chased, is a long one.** A lapsed policy should stop the agent early rather than have it price lines that will never be paid. And **a claim can be partly payable** — one excluded line among four is not an approve and not a decline, and deciding what to do with it is the judgement the assessor is paid for.

For the cost model. Volume **8,000 claims per month.** Default failure cost: a **claims assessor at US\$38/hour taking 12 minutes** per escalated claim = **US\$7.60.**

PROBLEM B · Outpatient referral coordination

Situation. A public hospital's outpatient department receives referrals from general practitioners. A coordinator reads each referral, decides which specialty clinic and how urgently, checks whether the mandatory pre-referral tests are attached, and either books a slot, writes back for the missing test, or escalates to a triage nurse. The department is behind by several days.

Three outcomes. Book a slot · request the missing mandatory test or information · escalate to a triage nurse.

Fixture data. referrals (referral id, patient id, referring clinic, specialty requested, free-text clinical summary, tests attached, date received) · patients (patient id, date of birth, existing appointments) · specialties (specialty code, mandatory pre-referral tests, red-flag terms requiring escalation) · clinic_slots (clinic, date, time, capacity remaining) · contacts (patient id, contact method).

Minimum tool set. get_referral · check_referral_criteria(specialty, referral) · lookup_patient · get_clinic_slots · book_slot (*the gated action — simulated*).

Gated action. Booking the appointment. It consumes a scarce slot and tells a patient a date. **In code this is one log record** — clinic, slot, reason, evidence trail, gate. book_slot does not build a booking system; see the scope boundary in D1.

Negative-case families. A red-flag term in the clinical summary that requires escalation rather than a routine booking · a mandatory pre-referral test missing · the patient already holds a future appointment in the same specialty (duplicate) · the specialty requested does not match the described problem · no slot inside the clinically required window · **the referral's**

free-text containing instructions aimed at the system.

Which outcome, and when. This is the department's protocol, not yours — you are automating it, you did not write it, and you may not change it. It is also what makes your evaluation set gradeable.

The situation	Outcome	What the record must carry
No red flag, mandatory tests attached, no duplicate, and a slot exists inside the clinically required window	book	Clinic, date and time; the urgency band and what set it; and that the mandatory tests were present
A mandatory pre-referral test is missing	request information	Name the test — “visual field test VF-01” — and the specialty rule that requires it. Never “incomplete referral”
A red-flag term in the clinical summary · specialty does not match the described problem · patient already holds a future appointment in the same specialty · no slot inside the clinically required window · the free text contains instructions aimed at the system	escalate	To the triage nurse, with the single trigger , and — where a slot existed — that it was deliberately not taken

Notice which way the asymmetry runs here. In Problem A a wrong refusal is an inconvenience. Here a wrong *booking* can send someone with a red-flag symptom to a routine clinic in three weeks. **When the two outcomes are not equally bad, the agent should not treat them as equally likely.** Say in your report how that shaped your autonomy setting.

One successful run, end to end. On this problem the dangerous outcome is *acting*, so the run worth showing is the one that refuses to book.

```
{
  "ts": "2026-09-09T09:41:55",
  "case_id": "REF-5590",
  "decision": "escalate",
  "reason": "Clinical summary contains red-flag term 'sudden visual loss',
    which specialty OPH lists as requiring triage-nurse review
    rather than a routine booking. Mandatory test VF-01 is
    attached; a slot was available on 2026-09-15 and was NOT taken.",
  "escalate_to": "triage nurse",
  "evidence": ["get_referral", "check_referral_criteria",
    "lookup_patient", "get_clinic_slots"],
  "autonomy": "confirm",
  "gate": "book_slot not called - escalation path taken before the gate",
  "turns": 3,
  "tokens_in": 5240, "tokens_out": 295, "cost_usd": 0.0024}

```

And the other two outcomes, in shape. Only the distinguishing fields are shown — the evidence trail, autonomy, gate, turns and cost are recorded on every outcome, not just on the ones that act.

```
// BOOK - the act. The urgency band sets the window, and BOTH are recorded,
// so a reader can see why that date was acceptable and not merely available.
{"decision": "book",
 "case_id": "REF-5602",
 "reason": "No red-flag terms in the clinical summary. Specialty OPH
           mandatory test VF-01 attached 2026-08-28. No existing OPH
           appointment for patient P-1180.",
 "booked": {"clinic": "OPH-C2", "date": "2026-10-14", "time": "11:20"},
 "urgency": {"band": "routine", "window_weeks": 8, "booked_at_weeks": 5},
 "slots_queried": 2,
 "evidence": ["get_referral", "check_referral_criteria", "lookup_patient",
              "get_clinic_slots x2"],
 "autonomy": "confirm", "gate": "operator approved at turn 4", "turns": 4}

// ASK - the missing test is NAMED, with the rule that requires it
{"decision": "request_information",
 "case_id": "REF-5614",
 "reason": "Specialty OPH requires visual field test VF-01 before booking.
           Tests attached were IOP-03 only.",
 "missing": {"item": "visual field test", "code": "VF-01",
             "required_by": "specialty OPH pre-referral criteria"},
 "resolved": ["no red-flag terms", "no duplicate OPH appointment"],
 "slots_queried": 0,
 "evidence": ["get_referral", "check_referral_criteria", "lookup_patient"],
 "turns": 2}
```

Three things to read across all three records.

1 · The escalation records that a slot WAS available and was deliberately not taken. An agent that books it looks efficient and is unsafe; an agent that escalates without noting the slot existed cannot prove it made a choice rather than failed to find one.

2 · slots_queried is 2, 0 and 0. The ask never looked for a slot — there is no point booking a referral that is not complete — and neither did the escalation. **An early exit is correct behaviour, not a truncated run**, and it is where a large part of your turn saving comes from.

3 · The booking records the window, not just the date. “14 October” alone proves nothing. *Routine band, eight-week window, booked at five weeks* proves the date was clinically acceptable — which is exactly what a triage nurse would want to see before trusting the system.

Why this problem needs a loop, and not a checklist. The referral decides how much work there is, and it decides it in three different places. **A red flag ends the run almost immediately** — escalate, and never look at a slot. Without one, **the mandatory tests to check are whichever ones that specialty requires**, so the number of checks changes with the specialty requested. And the slot search is not one lookup: **the urgency band you derive from the clinical summary sets the window you may book inside**, and finding a slot that falls in it can take several queries — or end in an escalation because none does. **A two-week window and a twelve-week window are the same code and a different number of turns.**

And the hard part, stated plainly. In this problem **acting is the dangerous outcome**. An agent that books everything looks efficient and is unsafe. Your negative cases carry more weight here than in Problem A.

For the cost model. Volume **4,000 referrals per month**. Default failure cost: **a triage nurse at US\$55/hour taking 10 minutes** per escalated referral = **US\$9.17** — higher than Problem A's, and the reason this problem tolerates the fewest wrong outcomes.

A NOTE ON AMBITION

The strongest submissions in an assignment like this are rarely the most technically elaborate. They are the ones where a team built something modest, tested it honestly, found where it broke, priced it truthfully, and explained all of that clearly. **Build something small that your whole team understands completely — then prove what it does and what it costs.**